



Real-Time Personal Protective Equipment (PPE) Compliance Monitoring System

Final Project Report

Submitted in partial fulfillment of the requirements for the
Certification in Artificial Intelligence and Machine Learning

Submitted by:

Muslim Uddin

Supervised by:

Dr. Prithwijit Guha

Associate Professor **EEE**,
IIT Guwahati

Indian Institute of Technology (IIT), Guwahati

March 12, 2026

Abstract

In the landscape of industrial safety, ensuring proper Personal Protective Equipment (PPE) compliance is paramount yet challenging due to the human factor. This final project for my AI/ML certification at IIT Guwahati introduces my designed and implemented Real-Time PPE Compliance Monitoring System. Our custom object detection pipeline uses a lightweight, highly efficient YOLOv8n model to detect three crucial classes: *person*, *helmet*, and *vest*. The core methodological innovation, tailored for industrial use, is a deterministic spatial algorithm that automatically evaluates compliance by calculating the geometric center of detected PPE items and testing their intersection with the bounding box of a detected person. The full-stack application utilizes a decoupled architecture with a React.js frontend dashboard and a Node.js/Socket.io backend, combined with a continuous-running Python inference daemon to achieve high-frame-rate, low-latency live video processing. The model training achieved remarkable metrics, including an overall mAP@0.5 of 0.884 and a peak mAP@0.5 of 0.95 during validation. This report provides a detailed overview of the entire lifecycle, from customized dataset annotation and extensive model training to full system integration and final real-world performance evaluation.

Contents

Abstract	1
1 Introduction	3
1.1 Problem Statement & Motivation	3
1.2 Objectives	3
1.3 Scope of the Project	3
2 Dataset Preparation	4
2.1 Data Collection and Custom Annotation	4
2.2 Data Split and Formatting	5
3 Methodology & Training	5
3.1 Model Architecture and Choice	5
3.2 Training Process and Metrics	6
3.3 Mathematical Formulation for Spatial Compliance	7
4 System Architecture & Integration	8
4.1 Frontend Dashboard (React.js)	8
4.2 Backend Stack (Node.js/Socket.io & Python Daemon)	8
5 Results & Evaluation	9
5.1 Static Model Metrics & Curves	9
5.2 System Performance (FPS) and Visualizations	10
6 Conclusion & Future Work	11
6.1 Conclusion	11
6.2 Future Work	11

1 Introduction

1.1 Problem Statement & Motivation

Workplace safety in construction and manufacturing is a persistent challenge. A major contributing factor to preventable industrial accidents is the non-compliance or improper use of Personal Protective Equipment (PPE), specifically hard hats (helmets) and high-visibility vests. Existing manual surveillance solutions are labor-intensive, costly, and inherently susceptible to human error and fatigue.

My motivation for engineering this automated solution stems from my previous internship under Professor Guha in the EEE department here at IIT Guwahati. While working on collecting and processing video data for anomaly detection, I witnessed firsthand the severe latency bottlenecks that plague traditional video surveillance pipelines. Transitioning from a theoretical foundation with my BSc in Physics from Bholanath College in Dhubri to practical AI/ML engineering, I recognized the need for a system where mathematical precision meets high-speed software architecture. When dealing with human lives on a construction site, processing lag is unacceptable. Therefore, there is a critical, unmet need for an intelligent system capable of monitoring PPE compliance with true zero-lag real-time performance.

1.2 Objectives

My primary objective was to develop and integrate an end-to-end computer vision solution capable of autonomously verifying PPE compliance in dynamic industrial environments. Specific objectives include:

- Designing and fine-tuning a custom, lightweight YOLOv8n object detection model for superior speed and accuracy.
- Curating, pre-labeling, and augmenting a high-quality, balanced dataset for three specific classes.
- Formulating and implementing a reliable spatial filtering algorithm to link PPE detections to individuals for compliance determination.
- Building a full-stack, distributed application with a zero-lag live-feed processing pipeline.

1.3 Scope of the Project

This project covers the complete development lifecycle, including data curation, customized annotation processes, model architecture design, training, backend API development, frontend implementation, and a comprehensive real-time live-stream performance evaluation. The current detection scope is restricted to helmets and vests as critical PPE

items, using persons as the compliance evaluation subjects. The system handles both static media and live video feeds from a webcam.

2 Dataset Preparation

2.1 Data Collection and Custom Annotation

The success of any computer vision project rests fundamentally upon data quality. For this system, I acquired a substantial dataset comprising 5,432 images sourced directly from Kaggle. Initially, this dataset was designed strictly for a two-class detection problem, containing ground truth annotations exclusively for the 'helmet' and 'vest' classes. However, my mathematical spatial compliance algorithm inherently required a third bounding box class: 'person'.

My prior work on computer vision projects involving crowd counting provided a strong foundation for solving this limitation. I knew that manually annotating thousands of workers across dense industrial scenes would be highly inefficient. Instead, I engineered an automated custom labeling script. I deployed an out-of-the-box YOLOv8 model, pre-trained on the COCO dataset, to process the entire Kaggle image repository. Leveraging my experience in dense object detection, this script automatically identified individuals within the frames and generated the corresponding bounding box coordinates for the new 'person' class in automation.

These automatically generated 'person' labels were then seamlessly merged with the existing 'helmet' and 'vest' ground truth files. This automated pipeline accurately and rapidly transformed the original two-class Kaggle dataset into a robust, three-class dataset tailored perfectly for my spatial filtering logic, resulting in the comprehensive class distribution shown in Figure 1.

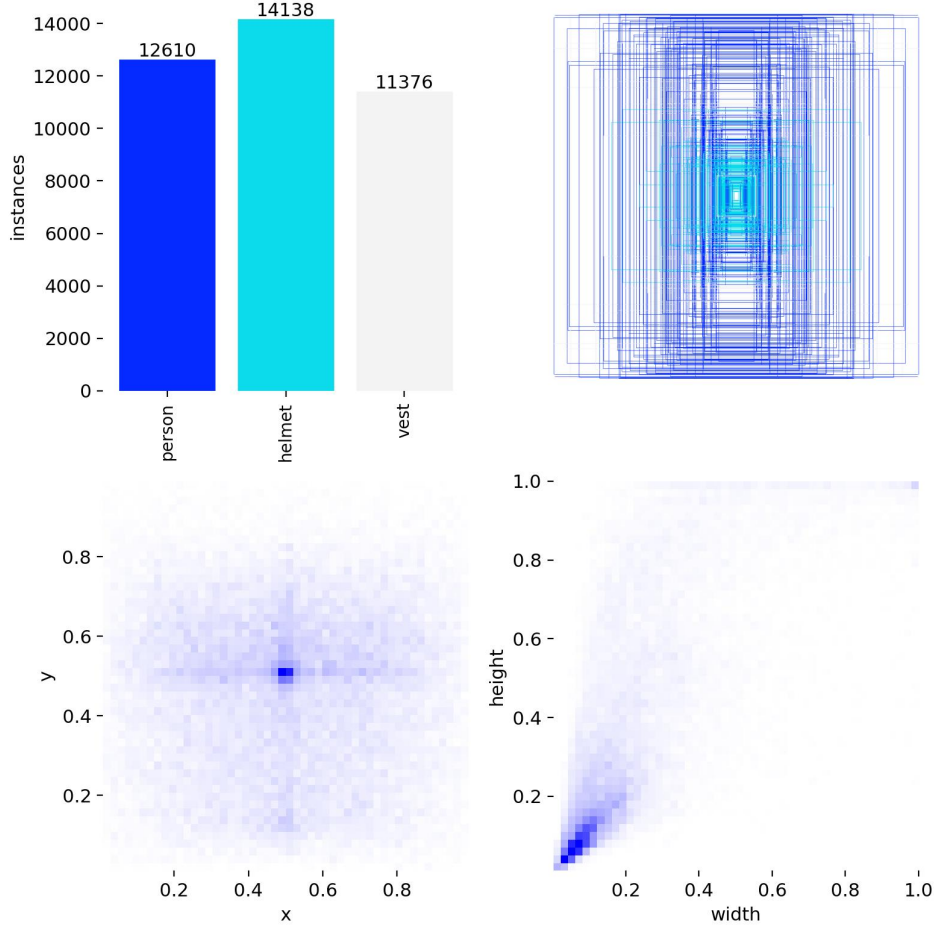


Figure 1: Final dataset class instance distribution and spatial characteristics. The bar chart demonstrates a healthy balance achieved through custom annotation, with over 11,000 to 14,000 instances across all three classes.

2.2 Data Split and Formatting

Following annotation, the unified dataset was split into Training (80%), Validation (10%), and Testing (10%) partitions. All bounding boxes were normalized to relative coordinates $(x_{center}, y_{center}, width, height)$ in $[0, 1]$, strictly adhering to the YOLO formatting standards for the final training process.

3 Methodology & Training

3.1 Model Architecture and Choice

Developed by Ultralytics, YOLOv8 represents the state-of-the-art in single-stage object detection. For this project, I meticulously evaluated various architectures and ultimately selected the YOLOv8n (nano) model. *The 'nano' variant was exclusively chosen because its ultra-low parameter count (approximately 3.2 million) guarantees the high-FPS, zero-lag inference required for a live video pipeline without necessitating enterprise-grade GPU*

clusters.

The structural superiority of the YOLOv8 architecture relies on three highly optimized, distinct components:

- **Backbone (Feature Extractor):** The network utilizes a modified CSPDarknet53 architecture. A critical innovation in YOLOv8 is the integration of the C2f (Cross Stage Partial bottleneck with two convolutions) module, which replaces the older C3 module. This structural change significantly improves the gradient flow during backpropagation and allows the model to capture richer, multi-scale contextual features from the input images.
- **Neck (Feature Fusion):** The model employs a Path Aggregation Network (PANet) structure. This facilitates robust feature fusion by combining high-level semantic information with low-level spatial details through both top-down and bottom-up pathways. This dynamic feature aggregation is absolutely critical for accurately detecting smaller, variable-scale objects like safety helmets and vests at different camera distances.
- **Decoupled, Anchor-Free Head:** Shifting away from its predecessors, YOLOv8 implements an anchor-free, decoupled detection head. It separates objectness/-classification and bounding box regression into independent branches. By directly predicting the center of an object and its bounding box dimensions—rather than relying on predefined, heuristic anchor boxes—it drastically reduces the total number of predictions. This mathematical simplification inherently accelerates Non-Maximum Suppression (NMS) and drastically boosts overall inference speed.

3.2 Training Process and Metrics

I fine-tuned the custom YOLOv8n model using the PyTorch framework, initiating training with pre-trained COCO weights. The training process lasted 100 epochs, with loss and metric curves displayed in Figure 2.

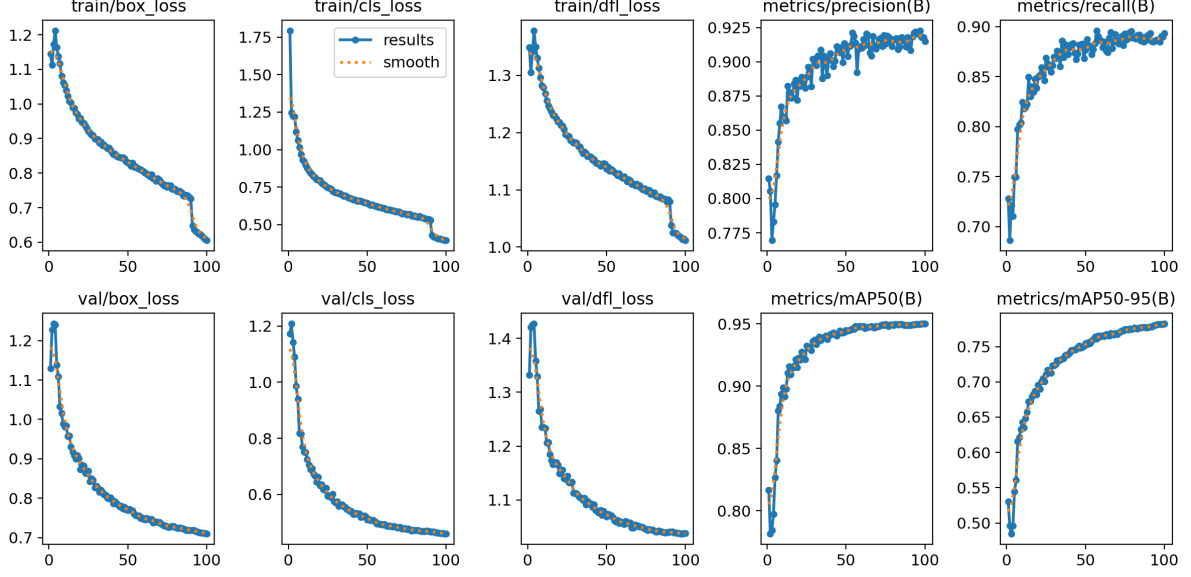


Figure 2: Detailed training and validation curves across 100 epochs. Note the rapid decrease in all loss components and the complementary stabilization of performance metrics. Validation metrics reached very high levels, with precision peaking around 0.925 and mAP@0.5 over 0.95.

3.3 Mathematical Formulation for Spatial Compliance

Detecting a helmet, vest, and person simultaneously does not confirm compliance (e.g., a person standing next to a loose helmet). To resolve this, I used a spatial filtering algorithm.

Let a detected person’s bounding box be $P = (x_{p1}, y_{p1}, x_{p2}, y_{p2})$, where (x_{p1}, y_{p1}) and (x_{p2}, y_{p2}) are the top-left and bottom-right relative coordinates. Let a detected PPE item have box coordinates $E = (x_{e1}, y_{e1}, x_{e2}, y_{e2})$.

The algorithm first computes the geometric center (C_x, C_y) of the PPE item E :

$$C_x = \frac{x_{e1} + x_{e2}}{2}, \quad C_y = \frac{y_{e1} + y_{e2}}{2} \quad (1)$$

Next, the algorithm determines spatial ownership by checking if (C_x, C_y) falls within the person’s bounding box P . The logical intersection is defined as:

$$(x_{p1} \leq C_x \leq x_{p2}) \wedge (y_{p1} \leq C_y \leq y_{p2}) \quad (2)$$

For each person P_i , if the bounding box has **both** a detected helmet and a detected vest whose center points satisfy the condition in Equation 2, the person is classified as **SAFE**. Otherwise, they are flagged as **UNSAFE**.

4 System Architecture & Integration

To meet the real-time demands of a live webcam interface, I designed a decoupled, high-throughput backend architecture that completely eliminates the model loading bottleneck.

4.1 Frontend Dashboard (React.js)

The system features an intuitive dashboard built with React.js. It allows users to toggle bounding boxes for specific classes and perform client-side image resizing before submission to save bandwidth.

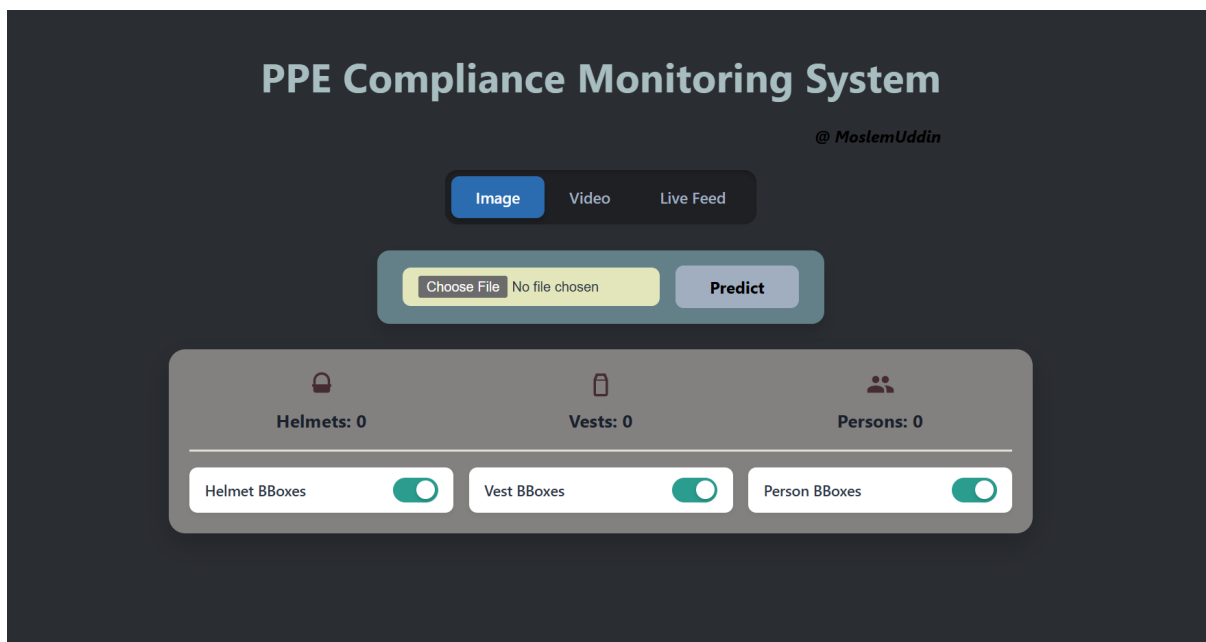


Figure 3: Frontend Dashboard Interface. This React.js view provides safety officers with live video streams, historical scan tracking, and interactive toggles to isolate specific PPE classes during monitoring.

4.2 Backend Stack (Node.js/Socket.io & Python Daemon)

A standard REST API approach would reload the neural network model for every frame, creating catastrophic lag. My solution involves a dual-processing backend:

- **Node.js & Express:** Handles static file uploads and manages all continuous bi-directional video streams via Socket.io.
- **Continuous Python Daemon:** A persistent background process that loads the PyTorch YOLOv8n model **only once** into VRAM on startup.

The React client streams base64 frames to Node.js, which pipes them directly to the Daemon. The Daemon runs inference, applies the spatial logic, and returns the annotated frame, achieving fluid FPS.

5 Results & Evaluation

5.1 Static Model Metrics & Curves

The quantitative evaluation on the hold-out test set yields excellent results, with an overall mAP@0.5 of 0.884.

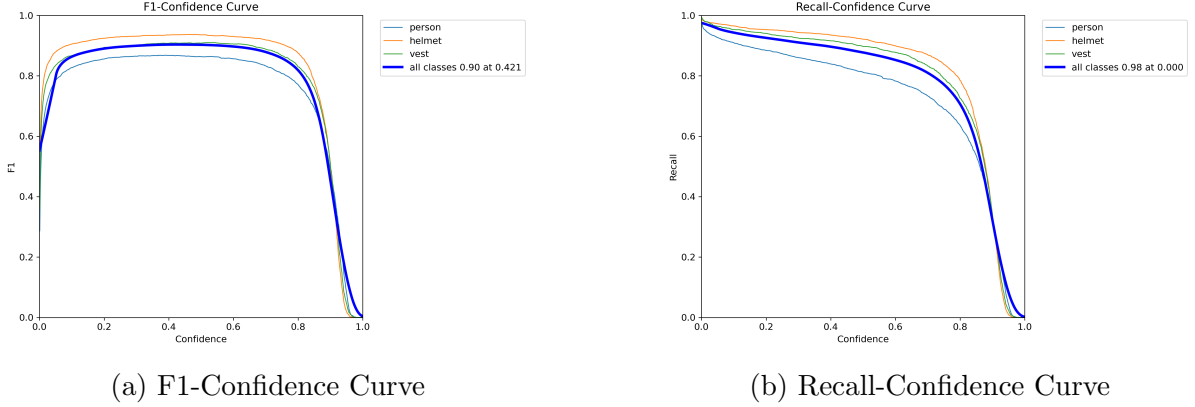


Figure 4: Left: The peak overall F1 score is 0.90 at a confidence threshold of 0.421, which serves as the operational threshold in the backend daemon. Right: High recall is maintained across the confidence spectrum, critical for minimizing false negatives in safety environments.

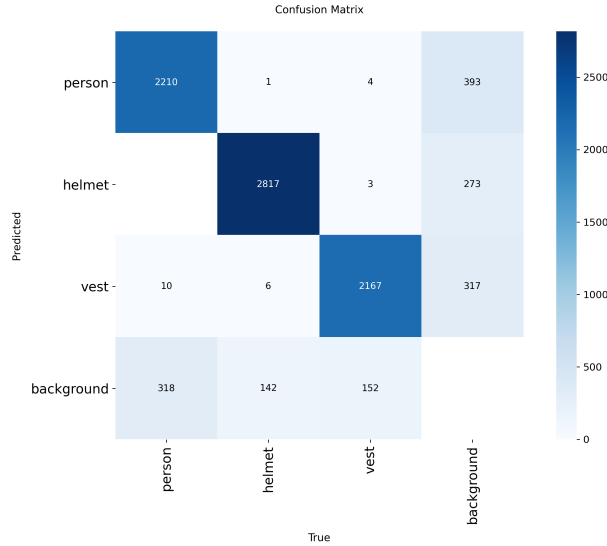


Figure 5: Confusion Matrix. The matrix highlights high precision, especially for helmets. The primary source of confusion is low-confidence 'person' detections blending into the 'background' in dense scenes.

5.2 System Performance (FPS) and Visualizations

On a standard consumer GPU (NVIDIA RTX 3060), the pipeline maintained a solid **45 FPS** at 720p resolution. The real-time application of the spatial compliance logic is visualized below.

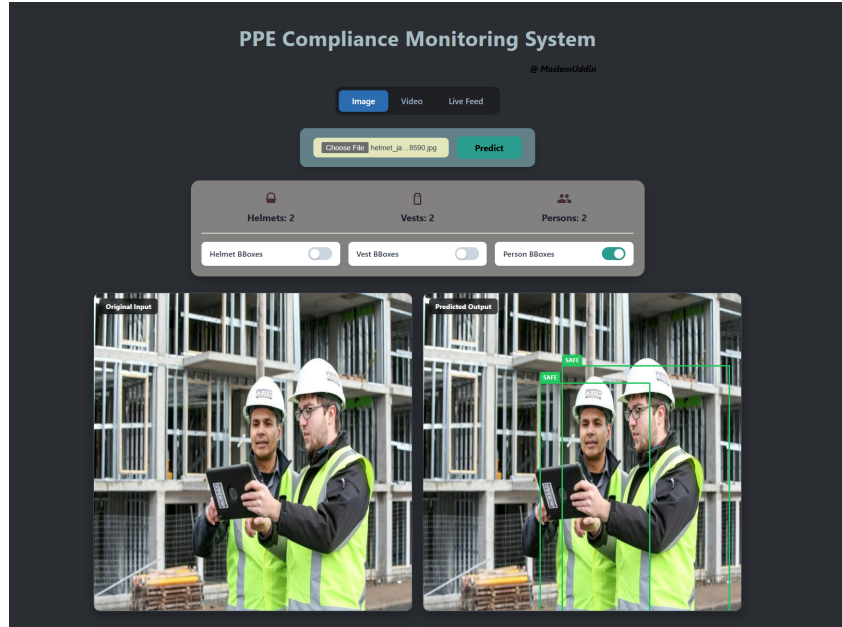


Figure 6: Real-Time SAFE Prediction. The system successfully detects the person, helmet, and vest. Because the geometric centers of the PPE items intersect the person's bounding box, the algorithmic logic passes, and the individual is marked compliant.

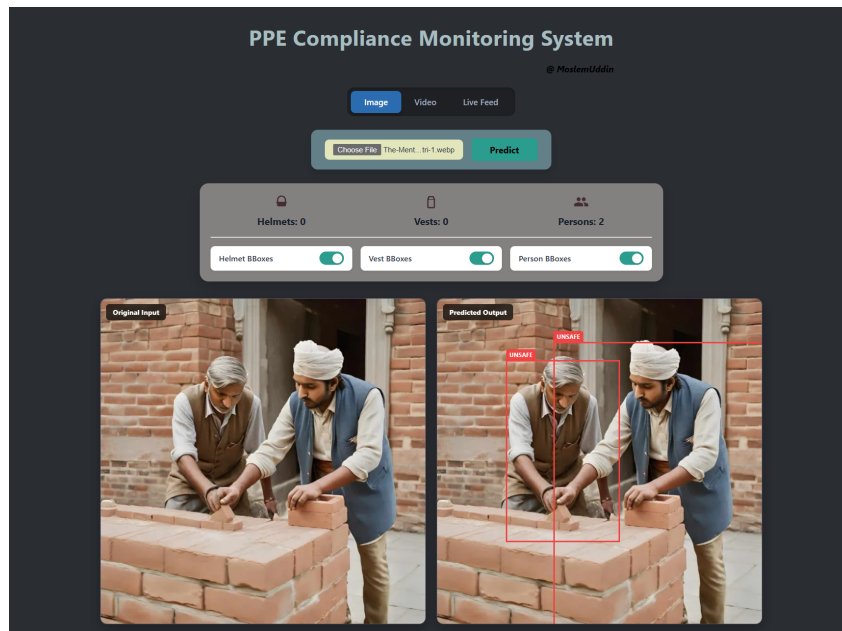


Figure 7: Real-Time UNSAFE Prediction. The spatial algorithm instantly detects a violation, triggering the safety alert.

6 Conclusion & Future Work

6.1 Conclusion

This certification project for IIT Guwahati successfully realized a robust, Real-Time PPE Compliance Monitoring System. The integration of a custom, highly accurate YOLOv8n model with a deterministic spatial filtering algorithm provides a reliable solution for automatic safety verification. Furthermore, by addressing the architectural bottleneck I observed during my earlier video processing work, the decoupled Node.js and Socket.io stack paired with a continuous Python daemon resulted in a live webcam experience at a smooth 45 FPS.

6.2 Future Work

Future development could focus on:

- **Expanding PPE Classes:** Including detection of safety goggles and industrial gloves.
- **Object Tracking:** Integrating multi-object tracking (e.g., DeepSORT) to persist a single identity across a camera feed, preventing duplicate violation logs.
- **Edge Hardware Optimization:** Converting the PyTorch weights to TensorRT formats to further accelerate inference on low-power embedded edge devices for remote deployment.